1 Data Overview and Initial Exploration

- **Dataset Source:** Kaggle House Prices: Advanced Regression Techniques
- **Initial Dimensions:** 1460 training samples (rows), 81 feature columns.
- **Data Type Distribution:** Includes 38 numerical features (`int64`, `float64`) and 43 categorical/text features (`object`).
- **Key Observations:** The dataset contains a significant amount of missing values. Furthermore, the large number of initial features makes direct application to linear models prone to overfitting and the curse of dimensionality.

2 Feature Engineering and Missing Value Imputation

To maximize data value and mitigate noise, we adopted a stratified treatment strategy for missing values rather than blind deletion:

2.1 Extracting Informative Missingness

- **Strategy:** Features with extremely high missing rates ($> 80\%$, e.g., `PoolQC`, `Alley`, `Fence`, `MiscFeature`) were transformed into binary features (1: Present / 0: Absent), creating new columns such as `HasPool`.
- **Decision Rationale:** This approach retains highly valuable signals for house price prediction (e.g., a luxury home possessing a pool) while resolving model instability caused by overly sparse original columns. The original high-missing-rate columns were subsequently dropped.

2.2 Numerical Feature Imputation

- **Strategy:** Implemented an automated imputation mechanism based on **Skewness**. The absolute skewness of each numerical feature was evaluated:
  - **Highly Skewed ($|\textbf{Skewness}| > 1.0$):** Indicates the presence of extreme outliers (e.g., ultra-luxury mansions). Imputed using the **median**.
  - **Roughly Symmetric ($|\textbf{Skewness}| \leq 1.0$):** Imputed using the **mean**.
- **Decision Rationale:** This dynamic strategy adapts to the underlying distribution of each individual feature, preserving the original statistical properties much more accurately than a uniform median imputation.

2.3 Categorical Feature Imputation

- **Strategy:** Abandoned uniform mode imputation in favor of a dual-track decision mechanism combining **statistical distribution** and **domain knowledge**:
  - **Extremely Low Missing Rate with Dominant Class (e.g., `Electrical`):** With a missing rate of only 0.1% and a mode dominance of 91.4%, this indicates random human omission. Imputed using the **mode**.
  - **Domain-Specific Correlated Missingness (e.g., `Garage` and `Bsmt` families):** Observed uniform missing rates across related features (5.5% for garage, 2.5% for basement). Domain knowledge dictates this represents the physical absence of the facility (i.e., "No Garage"), not missing data. Forced mode imputation would artificially synthesize non-existent amenities, thus these were imputed with **"None"**.
  - **High Missing Rate Features (e.g., `MasVnrType`, `FireplaceQu`):** Missing rates between 40%-60% indicate the general absence of these specific upgrades. Imputed with **"None"**.
- **Decision Rationale:** This prevents distribution distortion caused by blind mode imputation and ensures the engineered features strictly adhere to real-world real estate logic, thereby increasing the reliability of the predictive model.

3 Categorical Encoding

Since machine learning algorithms (Linear, Ridge, Lasso, SVR) cannot directly process text, strings must be converted into numerical representations:

- **Strategy:** Applied **One-Hot Encoding** utilizing `pd.get_dummies(drop_first=True)`.
- **Dimensionality Change:** After expanding over 40 categorical features, the total number of features expanded from 77 to 241.
- **Decision Rationale:** Using `drop_first=True` drops the first category level to bypass the **Dummy Variable Trap**. This ensures the stability of subsequent linear regression models and mitigates multicollinearity issues.

4 Target Variable and Feature Splitting

- **Feature Matrix (X) and Target Variable (y):** `SalePrice` was isolated as `y`, while the remaining 240 features constituted the feature matrix `X`.
- **Train/Test Split:** Utilized `scikit-learn` to split the dataset at an 80:20 ratio (Train: 80%, Test: 20%). A fixed `random_state` was configured to guarantee consistency and reproducibility in future model comparison benchmarks.

4.1 Baseline Linear Regression

4.2 Baseline Linear Regression (OLS)

To establish a rigorous performance benchmark for evaluating subsequent regularized models and advanced ensemble algorithms, this study first implemented a baseline model utilizing **Ordinary Least Squares (OLS)** linear regression.

4.2.1 Target Variable Transformation

Initial exploratory data analysis revealed that the target variable, `SalePrice`, exhibited a significant right-skewed distribution, as illustrated in the left panel of **Figure 1**. Such skewness can lead linear models to be overly sensitive to high-value outliers, potentially compromising generalizability.

To address this, a log-transformation was applied: $y = \log(1 + \text{SalePrice})$. As shown in the right panel of **Figure 1**, the transformed data approximates a normal distribution (Gaussian-like). This transformation satisfies the homoscedasticity and normality assumptions of linear regression and converts absolute price errors into relative percentage errors, ensuring more robust weight estimation across different price tiers.

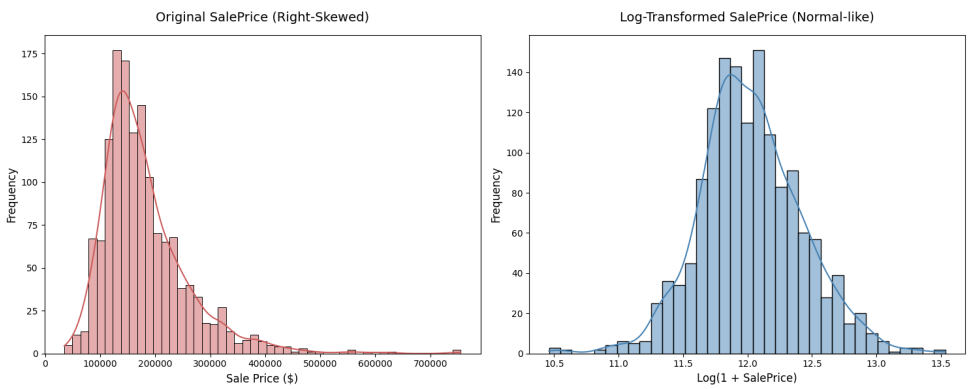


Figure 1: Distribution of SalePrice: Original skewed data (left) vs. Log-transformed normal-like data (right).

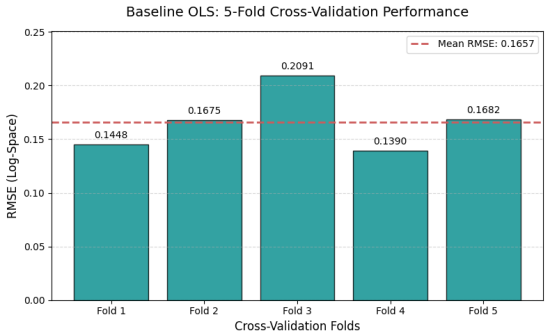
4.2.2 Model Evaluation and Visual Analysis

The baseline model was evaluated using **5-Fold Cross-Validation** on the development set (80% of the data) with a shuffle mechanism (`random_state=42`) to ensure stability.

Baseline Model Diagnostics:
To comprehensively evaluate the performance of the Baseline OLS model, we analyzed its

predictive distribution and stability through two complementary visualizations, as shown in Figure 2.

(a) Actual vs. Predicted Plot



(b) 5-Fold Cross-Validation Scores

Figure 2: Diagnostic visualizations of the Baseline OLS model, illustrating high variance and weight instability.

Based on the combined evidence from Figure 2, we identify the following challenges:

- **Catastrophic Underprediction and Weight Instability:** As illustrated in Figure 2a, the model suffers from extreme predictive failures. Specifically, a standard property with an **actual log value of $\sim$12.5** was severely underpredicted at a **log value of $\sim$9.75**. This magnitude proves that the OLS model overreacts to specific feature combinations in the 240-dimensional space, leading to erratic coefficient estimates.
- **Performance Fluctuation in CV:** Figure 2b confirms this instability, with RMSE scores fluctuating violently between Fold 3 (0.2091) and Fold 4 (0.1390). This sensitivity to data splits is a hallmark of **High Variance** and **Overfitting**.

These findings establish a definitive necessity for implementing **Ridge and Lasso Regression** to constrain the coefficient magnitudes and restore model robustness.

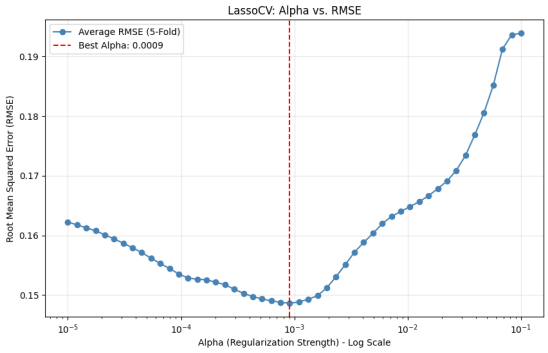
Metric	RMSE (Log-Space)
5-Fold CV Average RMSE	0.1657
Final Test RMSE	0.2048

Table 1: Performance Summary of Baseline Linear Regression

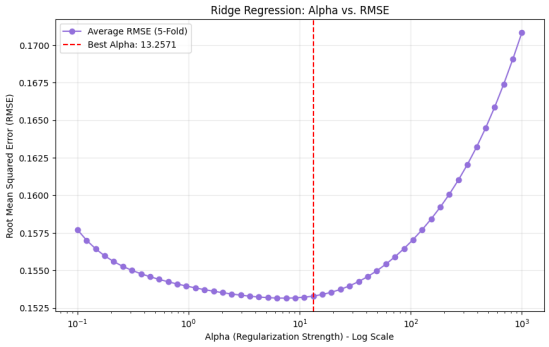
4.3 L1 and L2 Regularization (Lasso and Ridge)

4.4 Hyperparameter Tuning: Alpha Path Analysis

To determine the optimal regularization strength, we performed a log-scale grid search using 5-Fold Cross-Validation for both Lasso and Ridge models.



(a) LassoCV: Optimal $\alpha = 0.0009$



(b) RidgeCV: Optimal $\alpha \approx 13.25$

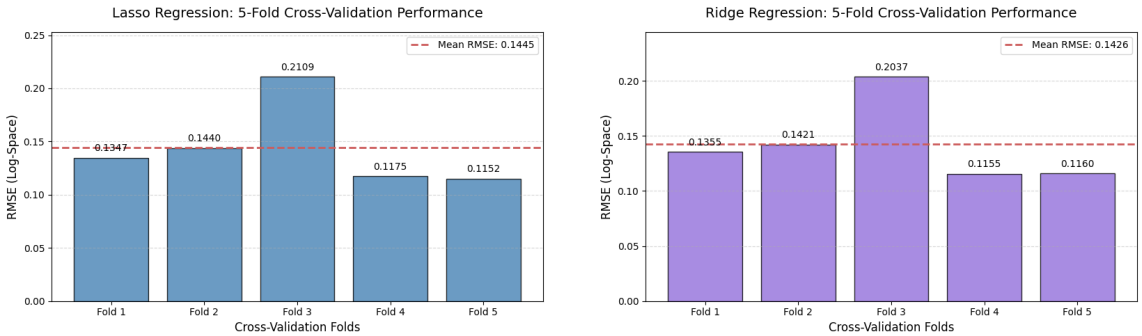
Figure 3: Hyperparameter tuning paths. The red dashed lines indicate the α values that minimize the mean Cross-Validation RMSE.

- **Under-regularization (Left side):** At extremely low α values, the penalty is negligible, and the models behave similarly to the unconstrained Baseline OLS. They suffer from high variance by capturing too much dataset noise, leading to elevated cross-validation errors.
- **Over-regularization (Right side):** Conversely, as α increases excessively, the models face severe penalties. Critical feature weights are shrunk to zero (Lasso) or near-zero (Ridge), resulting in high bias and underfitting, which causes the RMSE to spike drastically.
- **The Optimal Sweet Spot (Red dashed line):** The global minimum on each curve represents the optimal balance between bias and variance. Notably, Lasso achieves its optimum at a highly sensitive, small scale ($\alpha = 0.0009$), whereas Ridge optimizes at a vastly larger scale ($\alpha \approx 13.25$). This discrepancy is driven by their mathematical

formulations: the L1 norm (absolute sum) in Lasso applies a much harsher shrinkage per unit of α compared to the L2 norm (squared sum) in Ridge.

By systematically selecting these specific α parameters, we ensure the models effectively filter out idiosyncratic noise while retaining robust predictive signals.

4.5 Model Stabilization: Cross-Validation Performance



(a) Lasso CV (Mean RMSE: 0.1445)

(b) Ridge CV (Mean RMSE: 0.1426)

Figure 4: Cross-Validation performance of regularized models.

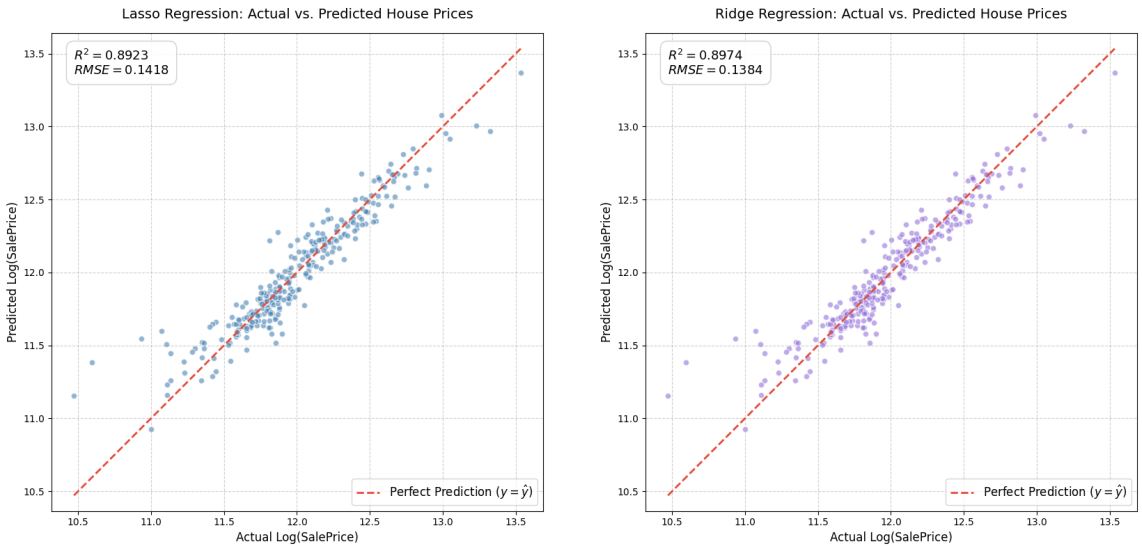
- As shown in **Figure 4**, both models achieved lower overall mean RMSEs (Lasso: 0.1445, Ridge: 0.1426) compared to the unregularized baseline, successfully dampening the extreme predictive fluctuations across most data splits.
- However, a critical observation must be made regarding the persistent instability in **Fold 3**. Despite the application of optimal L1 and L2 penalty terms, Fold 3 exhibits a disproportionately high error (Lasso: 0.2109, Ridge: 0.2037) relative to the other folds, which tightly cluster between 0.11 and 0.14.
- This prominent residual variance reveals a fundamental limitation: while regularization effectively constrains coefficient magnitudes to prevent overfitting, it cannot fully mitigate variance if the underlying data slice contains severe outliers or structural anomalies. The resilient spike in Fold 3 suggests the presence of complex, non-linear relationships within that specific validation subset that fundamentally violate the assumptions of linear models, warranting future exploration with robust, non-linear algorithms (e.g., Tree-based models).

4.6 Outlier Mitigation in Individual Predictions

While the cross-validation metrics reveal the macro-level behavior of the regularized models across different data splits, analyzing the Actual vs. Predicted scatter plots provides crucial

micro-level insights into individual property valuations.

This visual diagnostic reveals the true triumph of regularization: the elimination of catastrophic weight instability.



(a) Lasso Regression

(b) Ridge Regression

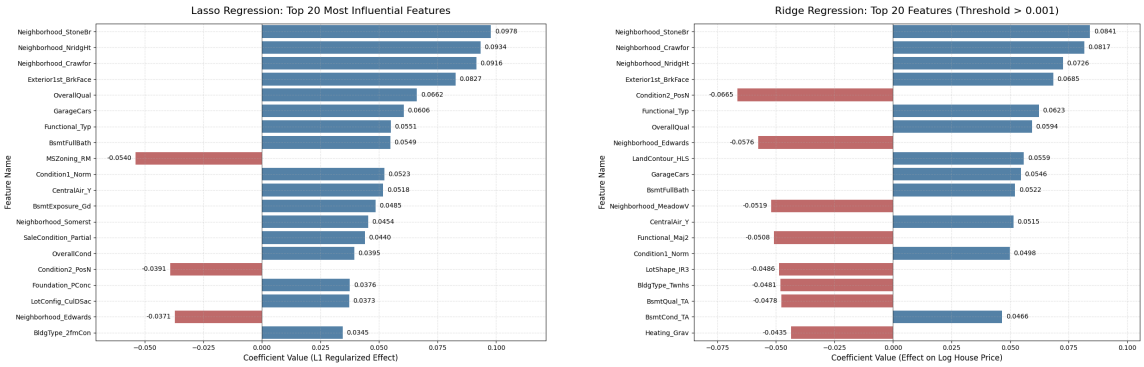
Figure 5: Actual vs. Predicted distributions for regularized models. Both techniques successfully eliminate the severe underprediction outliers observed in the baseline OLS.

As illustrated in **Figure 5**, both Lasso and Ridge models successfully bounded the predictions closely to the 45-degree reference line. Most notably, the catastrophic underprediction present in the Baseline OLS model—where a standard property (Actual Log $\sim$12.5) was severely underestimated to a Log value of $\sim$9.75—has been completely eradicated.

By penalizing large coefficients, both L1 and L2 regularizations acted as a crucial safety net. Even if they could not perfectly flatten the variance across all data folds (as seen in Section 4.5), they effectively prevented the model from overreacting to idiosyncratic noise, ensuring that no individual property suffers from extreme valuation errors.

4.7 Feature Insights: Sparsity vs. Shrinkage

While both models improved predictive stability, their handling of the 240-dimensional feature space differed fundamentally due to their distinct penalty terms.



(a) Lasso: Feature Selection (Sparsity)

(b) Ridge: Coefficient Shrinkage

Figure 6: Top 20 most influential features.

Model	Penalty	Total Features	Retained	Suppressed	Criteria
Lasso	L1	250	**92**	158	Coef = 0
Ridge	L2	250	**217**	33	$\lvert\text{Coef}\rvert < 0.001$

Table 2: Quantitative comparison of feature selection and shrinkage effects between Lasso and Ridge regression.

4.7.1 Quantitative Impact: Sparsity vs. Shrinkage

As quantified in **Table 2**, the fundamental difference between L1 and L2 regularization is explicitly revealed in their final active feature counts. Lasso effectively acts as a strict automated feature selector; it aggressively forces the coefficients of 158 less informative or highly collinear features to exactly zero, retaining a sparse subset of only 92 critical predictors. In contrast, Ridge employs continuous coefficient shrinkage. By retaining 217 features with absolute weights greater than 0.001, Ridge demonstrates its mathematical preference to distribute the penalty evenly across correlated variables rather than eliminating them entirely.

4.7.2 Qualitative Consensus on Market Drivers

Despite these opposing mathematical mechanics, **Figure 6** demonstrates that both models reach a striking consensus on the most impactful drivers of property value.

- **Top Value Additions:** Premium neighborhood indicators, such as `Neighborhood_StoneBr` and `Neighborhood_NridgHt`, along with visual and structural quality metrics like `OverallQual`, consistently dominate the top positive coefficients.

- **Top Value Detractions:** Similarly, adverse conditions like `Condition2_PosN` or specific lower-valued areas such as `Neighborhood_Edwards` receive the heaviest negative penalties.

Ultimately, this dual analysis highlights the strategic trade-off in regularization: Lasso provides a highly interpretable, streamlined model by isolating the absolute strongest signals, whereas Ridge preserves the nuanced, collective predictive power of the entire feature space while safely dampening its overall variance.

4.7.3 Overall Performance: Regularized Models vs. Baseline

To conclusively evaluate the effectiveness of the regularized models, we compared their cross-validation stability, final predictive accuracy on the test set, and real-world business interpretability.

Since our models were trained on a log-transformed target variable ($y = \log(1+\text{SalePrice})$), the resulting Log-RMSE inherently approximates the relative error. To translate this statistical variance into an intuitive business metric, we explicitly converted the Log-RMSE into a **Mean Percentage Error** using the following exponential transformation:

$$\text{Mean Percentage Error} = \left(e^{\text{RMSE}_{\log}} - 1\right) \times 100\% \tag{1}$$

This formula mathematically maps the variance observed in the logarithmic space directly to the expected percentage deviation in real-world dollar valuations.

Model	**CV RMSE (Log)**	**Test RMSE (Log)**	**MPE**
Baseline OLS	0.1657	0.2048	22.73%
Lasso (L1)	0.1445	0.1418	15.23%
Ridge (L2)	**0.1426**	**0.1384**	**14.84%**

Table 3: Performance comparison across linear models.

As summarized in **Table 3**, the application of regularization significantly stabilized the training process and improved generalization.

More importantly, evaluating the models using Mean Percentage Error reveals the tangible real-world impact of our feature engineering and hyperparameter tuning. The Baseline OLS model suffers from an unacceptable average error margin of 22.73%, rendering it unsuitable for practical real estate valuation. In contrast, both regularized models successfully

compressed this margin, with Ridge regression achieving the highest accuracy, bringing the average prediction error down to a much more reliable 14.84%.

5 Advanced Ensemble Modeling: Gradient Boosting Regressor (GBR)

5.1 The Transition to Non-Linear Ensembles

While regularized linear models (Lasso and Ridge) successfully mitigated variance and provided high interpretability, they remain fundamentally constrained by their linear assumptions. Real estate valuation inherently involves complex, non-linear relationships (e.g., the diminishing marginal utility of living area) and nested feature interactions (e.g., the combined effect of neighborhood and overall quality). To capture these intricate patterns without manual polynomial expansion, we transitioned to a non-linear ensemble paradigm using the **Gradient Boosting Regressor (GBR)**.

5.2 Hyperparameter Optimization: GridSearchCV

The performance of a Gradient Boosting Regressor (GBR) is highly sensitive to its hyperparameter configuration. To systematically identify an effective model structure, we initially employed a **Grid Search Cross-Validation (GridSearchCV)**. The search space was defined across four critical dimensions governing the ensemble's learning dynamics and tree architecture.

Table 4 summarizes the predefined search intervals, the optimal parameter combination identified by the grid, and the resulting cross-validation performance.

Hyperparameter	**Search Space**	**Best Parameter**
`n_estimators`	[500, 1000]	1000
`learning_rate`	[0.01, 0.05, 0.1]	0.05
`max_depth`	[3, 4, 5]	3
`subsample`	[0.8, 0.9]	0.9

Table 4: GridSearchCV parameter space, optimal configuration.

Model	CV RMSE (Log)	Test RMSE (Log)	MPE
GBR(GridSearchCV)	0.1264	0.1353	14.49%

Table 5: GBR(GridSearchCV) resulting performance.

Despite yielding a strong Cross-Validation RMSE of 0.1264, relying on GridSearchCV presents notable theoretical and computational limitations. Because Grid Search operates strictly on discrete, predefined intervals, it suffers from a inherent "blind spot" effect—it cannot guarantee finding the true global optimum if the ideal parameter values lie between the specified grid points.

Furthermore, exhaustive search scales poorly with dimensionality. Evaluating every possible combination in our specified grid requires computing $2 \times 3 \times 3 \times 2 = 36$ distinct model configurations (amounting to 180 total fits across 5 folds). This incurs substantial computational cost and limits our ability to expand the search boundaries. To resolve this inefficiency and enable a more continuous, stochastic exploration of the parameter space, we subsequently transitioned to **RandomizedSearchCV**.

5.3 Hyperparameter Optimization: RandomizedSearchCV

To comprehensively address the "blind spot" and the combinatorial explosion of the discrete grid, we implemented **RandomizedSearchCV**. The true advantage of this stochastic optimization lies in its ability to sample from continuous probability distributions rather than being confined to a finite list of predefined values.

Table 6 details the defined distributional search space and the refined optimal configuration discovered within our computational budget.

Hyperparameter	Search Range	Best Parameter
`n_estimators`	[300, 1500]	1308
`learning_rate`	[0.01, 0.10]	0.0243
`max_depth`	[3, 8]	3
`subsample`	[0.6, 1.0]	0.7246
`min_samples_split`	[2, 20]	16
`min_samples_leaf`	[1, 10]	5

Table 6: RandomizedSearchCV parameter distributions and the identified optimal configuration.

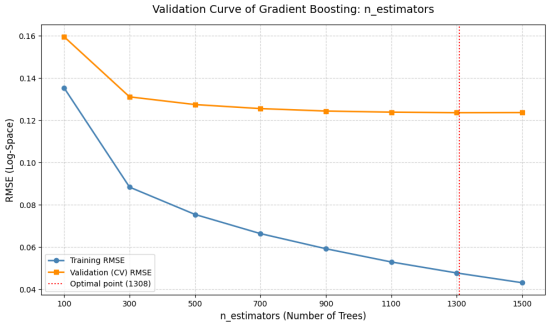
Furthermore, beyond achieving higher predictive accuracy, **RandomizedSearchCV** demonstrated superior computational efficiency. While the exhaustive Grid Search was forced to evaluate 180 total configurations to cover its sparse discrete grid, the Randomized Search was explicitly constrained to only 100 iterations. This highlights a profound advantage of stochastic optimization: by sampling intelligently from continuous distributions, the model successfully located a more optimal global minimum using significantly fewer computational resources.

Model	**CV RMSE (Log)**	**Test RMSE (Log)**	**MPE**
GBR(GridSearchCV)	0.1264	0.1353	14.49%
GBR(RandomizedSearchCV)	0.1249	0.1321	14.13%

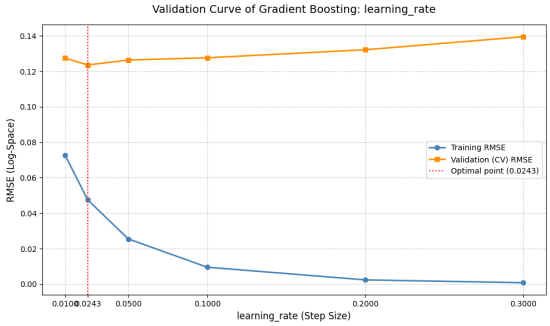
Table 7: GBR(GridSearchCV) resulting performance.

As demonstrated in Table 7, RandomizedSearchCV undeniably yields superior predictive performance. However, this observation naturally prompts a deeper analytical question: *how exactly were these optimal hyperparameters derived, and why does this specific combination outperform the grid search results?*

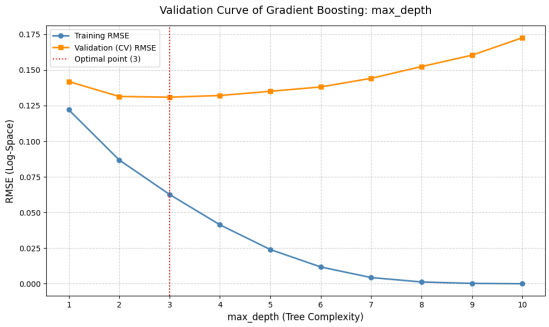
5.4 Validation Curve Analysis



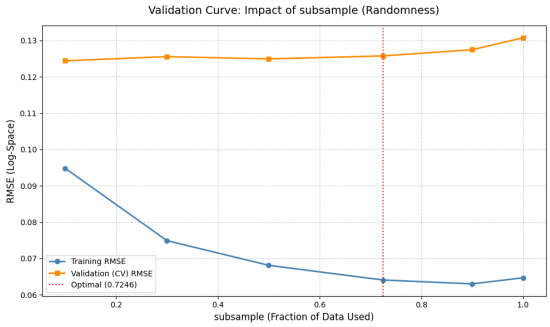
(a) Number of Estimators



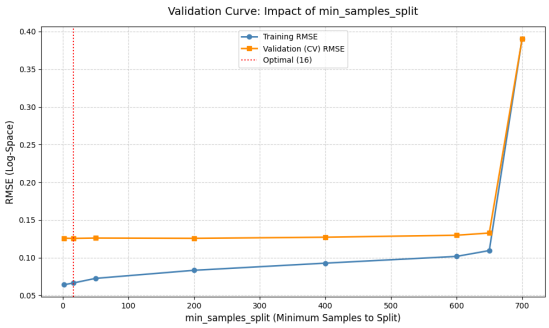
(b) Learning Rate



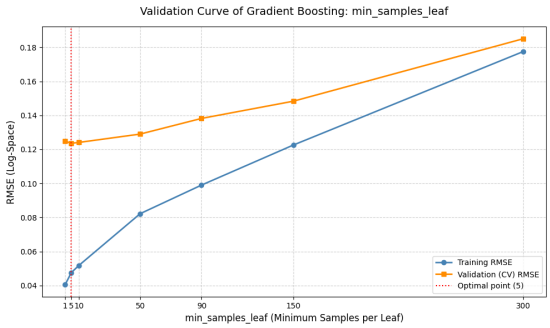
(c) Maximum Depth



(d) Subsample Ratio



(e) Min Samples Split



(f) Min Samples Leaf

Figure 7: Validation curves for the six key hyperparameters of the Gradient Boosting Regressor. The blue line represents the Training RMSE, while the orange line indicates the Validation (CV) RMSE. Vertical dotted lines mark the optimal parameters identified by the tuning process.

5.4.1 Impact of Ensemble Size (n_estimators)

As shown in Figure 7(a), the cross-validation error minimizes at `n_estimators = 1308`. Because Gradient Boosting builds trees sequentially to correct prior errors, our small step size (`learning_rate = 0.0243`) naturally requires a larger ensemble to achieve convergence.

However, the validation curve reveals a distinct plateau after approximately 700 trees, indicating **diminishing marginal returns**. By this point, the model has captured the primary predictive signals; subsequent learners contribute minimally, often fitting fine-grained residuals that represent irreducible noise.

While industrial applications involving large-scale data processing might truncate the ensemble at 700 trees to optimize efficiency, our dataset's small scale renders the computational overhead of 1308 trees negligible. Therefore, we retain the full 1308 estimators to strictly maximize predictive precision without compromising system performance.

5.4.2 Impact of Step Size (learning_rate)

The validation curve for the learning rate, illustrated in Figure 7(b), exhibits a classic U-shaped bias-variance tradeoff. We observe that the model achieves its minimum validation error at an optimal step size of `learning_rate = 0.0243`.

When the learning rate is set below this learning rate, both training and validation errors remain relatively high. This indicates a state of **underfitting**. Conversely, as the learning rate increases beyond the optimal point, the training error continues to approach zero while the validation error steadily rises. This divergence clearly demonstrates **overfitting**; an excessively large step size causes the algorithm to aggressively memorize local noise in the training data, thereby degrading its generalization capability on unseen data.

5.4.3 Impact of Tree Complexity (max_depth)

The validation curve for the maximum tree depth, shown in Figure 7(c), explicitly demonstrates the relationship between model capacity and the bias-variance tradeoff.

At lower depths (e.g., `max_depth = 1`), the ensemble suffers from **underfitting**. The base learners are too shallow to capture the underlying non-linear relationships within the housing data, resulting in elevated error rates across both the training and validation sets.

The validation error stabilizes in the region between depths 2 and 4, identifying `max_depth = 3` as the optimal structural complexity. Conversely, as the tree depth increases beyond 4, the training error rapidly approaches zero while the validation error steadily ascends. This pronounced divergence indicates **overfitting**. Trees that are permitted to grow too

deep construct overly complex decision boundaries, ultimately memorizing idiosyncratic noise and outliers in the training data rather than generalizing broader market trends.

5.4.4 Impact of Subsample Ratio (subsample)

This analysis evaluates the effect of `subsample` (the fraction of training instances randomly sampled to fit each individual base learner). As depicted in Figure 7(d), the model's performance traverses three distinct phases, empirically validating the mathematical necessity of Stochastic Gradient Boosting:

1. **Information Deficit Zone (0.1 – 0.3): Severe Underfitting**
   At excessively low subsampling ratios (e.g., 10%), the information available to each decision tree is highly fragmented. As observed, the training error reaches its global maximum in this zone, indicating a state of pronounced **high bias** and **underfitting** due to severe data restriction. Although this extreme stochasticity forcibly suppresses the validation error—creating a deceptively flat curve—this is merely a "false convergence" constrained by inadequate model capacity. In reality, the ensemble fails to adequately learn the deep, non-linear market features within the data, rendering configurations in this zone practically unusable.

2. **Performance Sweet Spot and Algorithmic Convergence (0.5 – 0.9)**
   The validation error reaches its minimum plateau within the 0.5 to 0.9 range. Visually, the validation error at 0.5 appears marginally lower than at 0.7246 (a difference falling within a negligible statistical margin of ~ 0.001). However, the `RandomizedSearchCV` algorithm converged on `subsample = 0.7246` as the empirical optimum. We validated and retained this algorithmically derived value based on two rigorous scientific considerations:

   - **Global vs. Local Optimum:** The value 0.7246 represents a global optimum identified by `RandomizedSearchCV` operating across a high-dimensional hyperparameter space (accounting for interactions with learning rate, depth, etc.). Evaluating a single parameter slice merely reveals a local minimum, which ignores these crucial synergistic effects.
   - **Information-Stochasticity Balance:** Retaining approximately 72% of the data preserves a robust amount of structural housing information while injecting sufficient randomness. This balance provides more stable predictions than aggressively discarding half of the dataset (i.e., 0.5).

3. **Loss of Stochastic Protection (1.0): Ensemble Overfitting**
   At `subsample = 1.0`, stochastic sampling is disabled, forcing all trees to train on the

exact same data. The resulting divergence—where training error continues downward while validation error rebounds—demonstrates that these highly correlated trees collectively memorize the same noise. Introducing stochasticity (e.g., 0.7246) effectively breaks this correlation, acting as a crucial regularization mechanism against ensemble overfitting.

5.4.5 Impact of Split Threshold: Asymmetric Growth (`min_samples_split`)

The validation curve for `min_samples_split` (Figure 7(e)) shows remarkable stability up to a threshold of 600, before experiencing a catastrophic underfitting cliff at 700. This phenomenon yields two profound insights into the model's architecture:

- **Asymmetric Splitting under Draconian Constraints:** Driven by impurity reduction, decision trees frequently isolate small subsets of outliers while retaining most instances in a dominant "trunk" node. A high threshold (e.g., 600) halts minority nodes but allows the trunk node to grow deeper, explaining the model's resilience.

- **Reverse-Engineering Topology:** The sudden collapse at 700 reveals that the maximum sample mass of the primary root nodes lies between 650 and 700. Surpassing this critical mass paralyzes the tree-building process at the root, reducing the ensemble to non-functional stumps.

5.4.6 Impact of Leaf Size: Theoretical Limits (`min_samples_leaf`)

At `max_depth = 3`, a perfectly symmetric tree contains up to $2^3 = 8$ leaves. Distributing our 1,168 training samples yields a theoretical average capacity of exactly 146 samples per leaf. The validation curve (Figure 7(f)) empirically corroborates this mathematical boundary through extreme stress testing. Setting `min_samples_leaf = 150` marginally exceeds this natural limit, triggering premature early stopping and an observable upward inflection in error. Furthermore, an aggressive threshold of 300 mathematically restricts the base learners to a maximum of three leaves ($1168 \div 300 \approx 3.89$), completely obliterating the depth-3 architectural advantage and causing severe underfitting. This destructive testing justifies our final permissive configuration (`leaf = 5`), which provides essential regularization against noise while fully unleashing the model's structural capacity.

6 Model Evaluation and Feature Importance

Having completed the comprehensive hyperparameter optimization, we can now conduct an in-depth comparison of the optimized Gradient Boosting Regression (GBR) model against the regularized linear baselines (Lasso and Ridge). By juxtaposing their internal stability and external generalization capabilities, we clearly establish the algorithmic superiority of

the ensemble approach before delving into the interpretability and feature importance of the model.

6.1 Comparative Performance: Stability and Generalization

(a) GBR Actual vs. Predicted Plot

(b) GBR 5-Fold Cross-Validation Scores

Figure 8: Diagnostic visualizations of the GB model, illustrating high variance and weight instability.

1. Internal Stability via Cross-Validation

Figure 8b illustrates the 5-fold cross-validation performance of our optimized GBR model. The ensemble achieved a highly competitive mean RMSE of 0.1237. More importantly, when compared to the linear baselines (e.g., Ridge, which exhibited a severe error spike of 0.2037 in Fold 3), the GBR model demonstrates remarkable fold-to-fold stability. The narrow variance across different data splits confirms that the sequential error-correction mechanism of boosting effectively mitigates the risk of overfitting to idiosyncratic subsets of the training data, yielding a highly robust architecture.

2. External Generalization on Unseen Data

The ultimate test of a predictive model lies in its generalization capability on unseen testing data. Figure 8a presents the Actual vs. Predicted scatter plot for the GBR model. The predictions cluster tightly along the red identity line ($y = \hat{y}$), culminating in an impressive R^2 score of 0.9064 and a test RMSE of 0.1321.

In contrast to Ridge regression ($R^2 = 0.8974$, RMSE = 0.1384), the Gradient Boosting ensemble exhibits noticeably less dispersion, particularly in the mid-to-high price ranges. This performance gap highlights a fundamental theoretical advantage: while Lasso and Ridge are constrained by linear assumptions, the tree-based GBR model successfully captures deep, non-linear feature interactions (e.g., the synergistic effect of neighborhood quality and living area) without requiring manual polynomial feature engineering.

Although noticeable deviations remain at the lower tail of the price distribution (i.e., lower-tier properties) likely due to unobserved idiosyncratic factors such as severe disrepair or distressed sales, the overall predictive precision confirms GBR as the superior model. With the predictive accuracy established, the following sections will open this algorithmic "black box" to analyze feature importance and masking effects.

6.2 The Dominance of Quality and Area

6.3 Linear vs. Non-Linear Discrepancy (Masking Effect)

7 Conclusion